

Operators in Programming

- **Operators:** Symbols used to perform actions on operands (values or variables).
 - Examples: =, +, typeof.
- **Operands:** Values or variables on which operators perform actions.
- **Operator Classification:**
 - **Unary:** Operates on 1 operand (e.g., typeof).
 - **Binary:** Operates on 2 operands (e.g., + for addition).
 - **Ternary:** Operates on 3 operands (e.g., conditional ?:).
- **Prefix, Postfix, Infix Operators:**
 - **Prefix:** Operator before operand (e.g., ++x).
 - **Postfix:** Operator after operand (e.g., x++).
 - **Infix:** Operator between operands (e.g., x + y).
- **Operator Context:** Different meanings based on operand types (e.g., + can be addition or string concatenation).

Assignment Operators

- **Basic Assignment:** = assigns the value of the right operand to the left.
 - Example: `const name = "Alice";`
- **Right to Left Evaluation:** Multiple assignments are evaluated right to left.
 - Example: `let year = 2050; let newYear = year = 2051;`
- **Other Assignment Operators:** Will be covered in related categories (arithmetic, logical, string).

Arithmetic Operators

- **Basic Arithmetic:**
 - Addition (+), Subtraction (-), Multiplication (*), Division (/), Remainder (%), Power (**).
 - Example: `console.log(5 + 2); // 7`
- **Operation Order:** Follows math precedence, parentheses can alter this order.
 - Example: `console.log(2 + 2 * 2); // 6`
- **Implicit Type Conversion:** JavaScript converts operands to Numbers unless it's string concatenation.
 - Example: `console.log("5" + 2); // "52"`

Unary Arithmetic Operators

- **Unary Operators:** + and - convert values to numbers, with - also negating the value.
 - Example: `let n = +"123"; // 123`
 - Example: `let n = -"123"; // -123`
 - Non-numeric strings result in NaN: `+"abcd" // NaN`.

Increment and Decrement Operators

- **Unary Increment (++) and Decrement (--):** Can be used as prefix or postfix.
 - **Prefix (++n):** Increment first, return new value.
 - **Postfix (n++):** Return value first, increment later.
- **Example:**

javascript

Copy code

```
let n1 = 5;
```

```
console.log(++n1); // 6
```

```
console.log(n1++); // 6, then n1 becomes 7
```

Compound Assignment Operators

- **Combined with Arithmetic:** Operators like +=, -=, *=, /=, %=, **= modify variables in place.
 - Example: `let x = 10; x += 5; console.log(x); // 15`
 - Example for **=: `x **= 2; // 25`

Logical Operators

- **Logical AND (&&):** Returns true if both operands are true.
 - Example: `console.log(true && false); // false`
- **Logical OR (||):** Returns true if at least one operand is true.
 - Example: `console.log(true || false); // true`
- **Logical NOT (!):** Negates the operand's boolean value.
 - Example: `console.log(!true); // false`

Logical Operators with Non-Boolean Values

- **NOT (!):** Converts non-Boolean values to Boolean before negation.
 - Example: `let x = "Alice"; console.log(!x); // false`

- **AND (&&) and OR (||)**: Return the first or second operand based on Boolean conversion.
 - **AND** returns the first falsy operand, or the second one if all are truthy.
 - **OR** returns the first truthy operand, or the second one if all are falsy.
 - Example: `console.log(0 && "test"); // 0`, `console.log(0 || "test"); // "test"`

Short-circuit Evaluation

- **AND (&&)**: Stops evaluation when the first operand is falsy.
- **OR (||)**: Stops evaluation when the first operand is truthy.
- **Example:**

javascript

Copy code

```
let x = 0, y = 0;
```

```
console.log(x++ && y++); // 0, y is not incremented
```

```
console.log(x); // 1, y remains 0
```

These quiz notes cover a range of operators in JavaScript, their usage, and how they behave with different operand types.